

1. Leer desde un archivo con Scanner

👉 **Scanner** no solo sirve para teclado, también para ficheros.

📌 **Ejemplo:**

```
import java.io.File;

import java.io.IOException;

import java.util.Scanner;


public class LeerArchivoScanner {

    public static void main(String[] args) {

        try {

            Scanner sc = new Scanner(new File("archivo.txt"));


            while (sc.hasNextLine()) {

                String linea = sc.nextLine();

                System.out.println(linea);

            }


            sc.close();


        } catch (IOException e) {

            System.out.println("Error al leer archivo");

        }

    }

}
```

🔍 **Idea clave:**

- `hasNextLine()` → controla si hay más líneas
 - `nextLine()` → lee línea completa
-

2. Comprobación de datos (evitar errores)

👉 Scanner permite validar antes de leer.

📌 Ejemplo:

```
while (sc.hasNextInt()) {  
    int num = sc.nextInt();  
    System.out.println(num);  
}
```

🔍 Explicación:

- Evita errores si el dato no es numérico
 - Muy útil en lectura de archivos o entradas externas
-

3. Representación de objetos (toString)

👉 Cuando haces:

```
System.out.println(objeto);
```

Java internamente hace:

```
objeto.toString();
```

📌 Ejemplo completo:

```
class Persona {  
    String nombre;  
  
    public Persona(String nombre) {
```

```
        this.nombre = nombre;
    }

    public String toString() {
        return "Nombre: " + nombre;
    }
}

Persona p = new Persona("Gabriel");

System.out.println(p);
```

Resultado:

Nombre: Gabriel

4. Ejemplo completo integrando Scanner + ficheros

👉 Leer datos del usuario y guardarlos en archivo

```
import java.io.FileWriter;

import java.io.IOException;

import java.util.Scanner;

public class GuardarDatos {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try (FileWriter fw = new FileWriter("datos.txt")) {

            System.out.print("Nombre: ");

            String nombre = sc.nextLine();
```

```
        System.out.print("Edad: ");

        int edad = sc.nextInt();

        fw.write("Nombre: " + nombre + "\n");

        fw.write("Edad: " + edad);

        System.out.println("Datos guardados");

    } catch (IOException e) {

        System.out.println("Error");

    }

    sc.close();

}

}
```

5. Buenas prácticas con Scanner

- ✓ Cerrar siempre `Scanner`
 - ✓ Usar `hasNext()` antes de leer
 - ✓ Evitar errores con `nextInt()` + `nextLine()`
 - ✓ Usar `nextLine()` para textos completos
 - ✓ Validar siempre entradas externas
-

Resumen

- `Scanner` → lectura de datos
- Puede leer:
 - teclado

- archivos
- `hasNext()` → evita errores
- `toString()` → define cómo se muestra un objeto
- Buenas prácticas → validar siempre entrada

EJERCICIOS

1. Leer archivo completo

Lee un archivo `datos.txt` y muestra su contenido por pantalla.

2. Leer archivo y contar líneas

Cuenta cuántas líneas tiene el archivo.

3. Leer números desde archivo

Lee un archivo con números y muéstralos solo si son enteros válidos.

4. Suma de números desde archivo

Lee números de un archivo y calcula la suma total.

5. Filtrar números positivos

Leer archivo y mostrar solo números positivos usando `hasNextInt()`.

6. Mostrar objetos Persona

Crear clase `Persona` con `toString()` y mostrar 3 objetos.

7. Simulación de listado de personas

Crear varias `Persona` y mostrarlas con `System.out.println()` (usando `toString()`).

8. Guardar datos en archivo

Pedir nombre y edad y guardarlos en `datos.txt`.

9. Registro de usuarios

Pedir varios usuarios hasta escribir "fin" y guardarlos en archivo.

10. Mini sistema de validación

Leer archivo con usuarios y comprobar si un nombre existe.